

OS Lab Exam — Complete Solutions with Explanations

How to Run Shell Scripts in Linux

```
bash

# Step 1: Create the file
nano script.sh          # opens a text editor in terminal

# Step 2: Give it execute permission
chmod +x script.sh

# Step 3: Run it
./script.sh
```

Key Shell Scripting Syntax Reference:

Syntax	Meaning
<code>#!/bin/bash</code>	"Shebang" — tells OS to use bash to interpret this file
<code>read var</code>	Read input from user into variable <code>var</code>
<code>\$var</code>	Access the value of <code>var</code>
<code>\$(command)</code>	Run a command and capture its output
<code>\$((expr))</code>	Arithmetic expression
<code>[condition]</code>	Test/condition evaluation
<code>-f file</code>	True if file exists and is a regular file
<code>for ((i=1; i<=n; i++))</code>	C-style for loop

Q1: Shell Script — Palindrome + Factorial

```
bash
```

```
#!/bin/bash
```

```
# — Line 1: Shebang —————  
# Tells the OS: "Use /bin/bash to run this script"  
# Without this, the OS might try to run it with sh or dash (different behavior)
```

```
# =====  
# PART 1 – PALINDROME CHECK  
# =====
```

```
echo "Enter a string to check for palindrome:"  
# echo: prints text to terminal (stdout)
```

```
read str  
# read: waits for user input, stores it in variable 'str'
```

```
rev_str=$(echo "$str" | rev)  
# echo "$str"          → prints the string  
# | (pipe)             → sends output of left command as input to right command  
# rev                  → reverses characters in a string (built-in Linux command)  
# $(...)               → command substitution: captures output into variable rev_str  
# Quoting "$str"       → important! prevents word-splitting if str has spaces
```

```
if [ "$str" = "$rev_str" ]; then  
# if [ ... ]           → starts a conditional test  
# "=" inside [ ]      → STRING equality (not arithmetic)  
# Always quote variables in [ ] to handle empty/space cases safely
```

```
    echo "$str is a Palindrome"  
else  
    echo "$str is NOT a Palindrome"  
fi  
# fi                  → closes the if block (if spelled backwards = fi)
```

```
# =====  
# PART 2 – FACTORIAL  
# =====
```

```
echo "Enter a number to find its factorial:"  
read n  
# User input stored in variable n
```

```
fact=1  
# Initialize factorial accumulator to 1 (0! = 1! = 1)
```

```
if [ $n -lt 0 ]; then  
# -lt = "less than" (arithmetic comparison inside [ ])
```

```
# Other comparisons: -gt (>), -le (<=), -ge (>=), -eq (==), -ne (!=)
    echo "Factorial is not defined for negative numbers"

elif [ $n -eq 0 ]; then
# elif = else if
    echo "Factorial of 0 is 1"

else
    for (( i=1; i<=n; i++ ))
    # C-style for loop: init ; condition ; increment
    # (( )) allows arithmetic directly (no $ needed inside)
    do
        fact=$(( fact * i ))
        # $(( )) = arithmetic expansion
        # Multiplies current fact by i and stores result back in fact
    done
    # done closes the for loop

    echo "Factorial of $n is $fact"
fi
```

Sample Run:

```
Enter a string: madam
madam is a Palindrome
Enter a number: 5
Factorial of 5 is 120
```

Q2: Shell Script — Word/Char/Line Count + Palindrome

```
bash
```

```
#!/bin/bash
```

```
# =====  
# PART 1 – FILE STATISTICS  
# =====
```

```
echo "Enter the filename:"  
read filename  
# Stores the filename (e.g., "notes.txt") in variable filename
```

```
if [ ! -f "$filename" ]; then  
# -f "$filename" → True if the path is a regular file  
# ! → NOT operator: inverts the condition  
# So: "if the file does NOT exist as a regular file..."  
    echo "Error: File '$filename' not found!"  
    exit 1  
    # exit 1 → terminate script with error code 1  
    # (exit 0 means success; non-zero means failure)  
fi
```

```
lines=$(wc -l < "$filename")  
# wc = word count utility  
# -l flag → count lines  
# < "$filename" → redirects file content as stdin to wc  
# $(...) captures the output (just the number, no filename)
```

```
words=$(wc -w < "$filename")  
# -w flag → count words (space-separated tokens)
```

```
chars=$(wc -c < "$filename")  
# -c flag → count bytes (characters in ASCII files)  
# Note: wc -m counts actual characters (handles multibyte/UTF-8)
```

```
echo "_____  
echo "File: $filename"  
echo "Lines      : $lines"  
echo "Words       : $words"  
echo "Characters  : $chars"  
echo "_____"
```

```
# =====  
# PART 2 – PALINDROME CHECK  
# =====
```

```
echo "Enter a string to check for palindrome:"  
read str
```

```
rev_str=$(echo "$str" | rev)
# Reverse the string using 'rev' command
# Pipe (|) connects echo output → rev input

if [ "$str" = "$rev_str" ]; then
    echo "'$str' IS a Palindrome"
else
    echo "'$str' is NOT a Palindrome"
fi
```

Sample Run (file: test.txt contains "Hello World\nFoo Bar"):

```
Lines      : 2
Words      : 4
Characters : 21
Enter a string: racecar
'racecar' IS a Palindrome
```

Q3: C Program — System Calls for File Operations

c

```

#include <stdio.h>    // printf, perror
#include <fcntl.h>    // open(), O_CREAT, O_RDONLY, O_WRONLY, O_TRUNC flags
#include <unistd.h>   // read(), write(), close(), lseek(), unlink()
#include <string.h>   // strlen()
#include <stdlib.h>   // exit()

int main() {
    int fd;           // File descriptor – integer handle returned by open()
    char buffer[100]; // Buffer to store data read from file
    char *data = "Hello, OS Lab! System Calls Demo.\n";
    int bytes;        // Return value of read/write

    // — STEP 1: CREATE & WRITE —————
    fd = open("demo.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    // open() → system call to open/create a file; returns file descriptor (int)
    // Arg 1 → filename
    // Arg 2 → flags combined with | (bitwise OR):
    //   O_CREAT → create file if it doesn't exist
    //   O_WRONLY → open for writing only
    //   O_TRUNC → truncate (clear) existing content to 0 length
    // Arg 3 → permissions (only used when O_CREAT is set):
    //   0644 → owner: read+write (6), group: read (4), others: read (4)
    // Returns → fd >= 0 on success; -1 on error

    if (fd < 0) {
        perror("open (write)");
        // perror() → prints "open (write): <system error message>" to stderr
        exit(1);
    }

    bytes = write(fd, data, strlen(data));
    // write() → system call to write to a file descriptor
    // Arg 1 → file descriptor
    // Arg 2 → pointer to data buffer
    // Arg 3 → number of bytes to write (strlen = length without null terminator)
    // Returns → number of bytes actually written; -1 on error

    printf("Wrote %d bytes to demo.txt\n", bytes);

    close(fd);
    // close() → system call to release the file descriptor
    // IMPORTANT: always close files to flush buffers and free resources

    // — STEP 2: READ —————
    fd = open("demo.txt", O_RDONLY);
    // O_RDONLY → open for reading only (no permissions arg needed)

```

```

if (fd < 0) {
    perror("open (read)");
    exit(1);
}

bytes = read(fd, buffer, sizeof(buffer) - 1);
// read() → system call to read from a file descriptor
// Arg 1 → file descriptor
// Arg 2 → buffer to store data
// Arg 3 → max bytes to read (sizeof - 1 leaves room for null terminator)
// Returns → number of bytes actually read; 0 = EOF; -1 = error

buffer[bytes] = '\0';
// Manually null-terminate the buffer so it's a valid C string
// read() does NOT add '\0' – that's a common bug if you skip this!

printf("Read from file: %s", buffer);
close(fd);

// — STEP 3: lseek (Random Access) —————
fd = open("demo.txt", O_RDONLY);

lseek(fd, 7, SEEK_SET);
// lseek() → system call to move the file offset (cursor)
// Arg 1 → file descriptor
// Arg 2 → offset in bytes
// Arg 3 → reference point:
// SEEK_SET → from beginning of file
// SEEK_CUR → from current position
// SEEK_END → from end of file
// This moves to byte 7 from the start

bytes = read(fd, buffer, 6);
buffer[bytes] = '\0';
printf("6 bytes from offset 7: \"%s\"\n", buffer);
// Should print: "OS Lab"

close(fd);

// — STEP 4: unlink (Delete File) —————
if (unlink("demo.txt") == 0) {
    // unlink() → system call to delete a file (removes directory entry)
    // Returns 0 on success, -1 on failure
    printf("File demo.txt deleted successfully.\n");
} else {
    perror("unlink");
}

```

```
}  
  
return 0;  
}
```

Compile & Run:

```
bash  
  
gcc q3_fileops.c -o q3_fileops  
./q3_fileops
```

Key Concept — File Descriptors:

- 0 = stdin, 1 = stdout, 2 = stderr (reserved)
 - `open()` returns the next available integer ≥ 3
 - All I/O system calls use this integer, not the filename
-

Q4: C Program — Process Info (Parent & Child)

c


```

#include <stdio.h>
#include <unistd.h>      // fork(), getpid(), getppid()
#include <sys/types.h>   // pid_t type definition
#include <sys/wait.h>    // wait()

int main() {
    pid_t pid;
    // pid_t → typedef for process ID (usually int)

    printf("Before fork: PID = %d\n", getpid());
    // getpid() → returns PID of the calling process

    pid = fork();
    // fork() → system call that DUPLICATES the current process
    // After this line, TWO processes exist running identical code
    // The difference: fork() returns DIFFERENT values in each:
    //   → In PARENT: returns the child's PID (a positive integer)
    //   → In CHILD : returns 0
    //   → On ERROR : returns -1 (no child created)

    if (pid < 0) {
        // Error case: fork failed (e.g., system process limit reached)
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        // — CHILD PROCESS BLOCK —————
        // This block runs ONLY in the child (fork returned 0)
        printf("\n— CHILD PROCESS —\n");
        printf("  My PID          : %d\n", getpid());
        // getpid() in child → child's own PID

        printf("  My Parent's PID: %d\n", getppid());
        // getppid() → returns PID of the parent process

        printf("  I am the child.\n");
        // Child exits naturally when this block ends
    }
    else {
        // — PARENT PROCESS BLOCK —————
        // This block runs ONLY in the parent (fork returned child's PID)
        wait(NULL);
        // wait(NULL) → parent BLOCKS here until ANY child terminates
        // This prevents the child from becoming a "zombie process"
        // (Zombie = child finished but parent hasn't called wait yet)
        // NULL → we don't care about the child's exit status
    }
}

```

```

    printf("\n—— PARENT PROCESS ——\n");
    printf("  My PID          : %d\n", getpid());
    printf("  My Child's PID   : %d\n", pid);
    // pid still holds the child's PID (returned by fork)

    printf("  Child has finished. Parent exiting.\n");
}

return 0;
}

```

Compile & Run:

```

bash

gcc q4_process.c -o q4_process
./q4_process

```

Sample Output:

```

Before fork: PID = 1234
—— CHILD PROCESS ——
  My PID          : 1235
  My Parent's PID: 1234
—— PARENT PROCESS ——
  My PID          : 1234
  My Child's PID  : 1235

```

Key Concept — fork() Memory Model:

- Parent and child get SEPARATE copies of memory (copy-on-write)
- Variables changed in child do NOT affect parent and vice versa

Q5: Producer-Consumer — With Count Variable + Semaphore + Mutex

c

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>    // pthreads: pthread_create, pthread_join, mutex
#include <semaphore.h>  // sem_t, sem_init, sem_wait, sem_post
#include <unistd.h>     // sleep()

#define BUFFER_SIZE 5    // Maximum items the buffer can hold at once
#define MAX_ITEMS 8     // Total items producer will produce

// — Shared Data (accessed by BOTH threads) —————
int buffer[BUFFER_SIZE]; // Circular buffer (shared memory)
int in = 0;               // Index where producer inserts next item
int out = 0;              // Index where consumer removes next item
int count = 0;            // Number of items currently in buffer (shared!)

// — Synchronization Primitives —————
sem_t empty;              // Counting semaphore: tracks EMPTY slots in buffer
sem_t full;               // Counting semaphore: tracks FULL slots in buffer
pthread_mutex_t mutex;    // Mutual exclusion lock for the critical section

// =====
void *producer(void *arg) {
    for (int i = 0; i < MAX_ITEMS; i++) {
        int item = rand() % 100;
        // Generate a random item (0-99) to produce

        sem_wait(&empty);
        // sem_wait → decrement empty by 1
        // If empty == 0 (buffer full), BLOCK here until consumer makes space
        // This prevents buffer overflow

        pthread_mutex_lock(&mutex);
        // Acquire the mutex lock
        // Only ONE thread can hold this lock at a time
        // Any other thread calling lock() will BLOCK → mutual exclusion

        // — CRITICAL SECTION START —————
        buffer[in] = item;
        // Place item at the 'in' index of the circular buffer

        in = (in + 1) % BUFFER_SIZE;
        // Advance 'in' with wrap-around using modulo
        // e.g., if BUFFER_SIZE=5 and in=4: next in = (4+1)%5 = 0

        count++;
        // Increment shared count — SAFE here because mutex is held
    }
}

```

```

// Without mutex: count++ is NOT atomic (read-modify-write race condition)

printf("[Producer] Produced: %3d | Buffer count: %d\n", item, count);
// — CRITICAL SECTION END —————

pthread_mutex_unlock(&mutex);
// Release the mutex – another thread can now acquire it

sem_post(&full);
// sem_post → increment full by 1
// Signals consumer that one more item is available

sleep(1); // Simulate time taken to produce
}
return NULL;
}

// =====
void *consumer(void *arg) {
    for (int i = 0; i < MAX_ITEMS; i++) {
        sem_wait(&full);
        // Wait until at least one item is in the buffer
        // Blocks if full == 0 (buffer empty)

        pthread_mutex_lock(&mutex);

        // — CRITICAL SECTION START —————
        int item = buffer[out];
        // Read item from the 'out' index

        out = (out + 1) % BUFFER_SIZE;
        // Advance 'out' with wrap-around

        count--;
        // Decrement shared count – SAFE because mutex is held

        printf("[Consumer] Consumed: %3d | Buffer count: %d\n", item, count);
        // — CRITICAL SECTION END —————

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
        // Signal producer that one more empty slot is available

        sleep(2); // Consumer is slower than producer (intentional)
    }
    return NULL;
}

```

```

}

// =====
int main() {
    pthread_t prod_thread, cons_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    // sem_init(sem, pshared, initial_value)
    // &empty      → pointer to semaphore
    // 0            → pshared=0 means shared between THREADS (not processes)
    // BUFFER_SIZE → initially all slots are empty

    sem_init(&full, 0, 0);
    // full starts at 0 → buffer starts empty (no items to consume)

    pthread_mutex_init(&mutex, NULL);
    // Initialize mutex with default attributes (NULL)

    pthread_create(&prod_thread, NULL, producer, NULL);
    // Creates a new thread running the producer() function
    // Arg1: thread handle, Arg2: attributes(NULL=default),
    // Arg3: function, Arg4: argument to pass to function

    pthread_create(&cons_thread, NULL, consumer, NULL);
    // Creates consumer thread

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);
    // join() → main thread WAITS for each thread to finish
    // Without join, main() might exit and kill all threads prematurely

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    // Cleanup: release resources

    return 0;
}

```

Compile & Run:

```
bash
```

```
gcc q5_producer_consumer_count.c -o q5 -lpthread
./q5
```

Q6: Producer-Consumer — Semaphore + Mutex (Clean Version)

This is the same bounded-buffer problem but without an explicit `count` variable — the semaphores alone track buffer state. This is the **canonical** solution.

c

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define MAX_ITEMS 8

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 1; i <= MAX_ITEMS; i++) {
        int item = i * 10;
        // Use sequential items (10, 20, ...) for clearer tracing

        sem_wait(&empty);    // Wait for empty slot
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        printf("[Producer] item %d → buffer[%d]\n", item, (in - 1 + BUFFER_SIZE) % B

        pthread_mutex_unlock(&mutex);
        sem_post(&full);    // Signal: one more full slot

        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= MAX_ITEMS; i++) {
        sem_wait(&full);    // Wait for item to be available
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        printf("[Consumer] item %d ← buffer[%d]\n", item, (out - 1 + BUFFER_SIZE) %

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);    // Signal: one more empty slot
    }
}

```

```

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Key Difference vs Q5: No `count` variable — the semaphore values themselves encode how many items are in the buffer (`full` = current items, `empty` = free slots). This is cleaner and preferred in production.

Q7: Dining Philosophers Problem

c


```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N      5      // Number of philosophers (= number of forks)
#define THINKING 0      // Philosopher state: thinking
#define HUNGRY  1      // Philosopher state: wants to eat
#define EATING  2      // Philosopher state: eating

// Macros for left and right neighbor indices (circular)
#define LEFT(i)  ((i + N - 1) % N)  // Left neighbor of philosopher i
#define RIGHT(i) ((i + 1) % N)      // Right neighbor of philosopher i

int state[N];          // Current state of each philosopher
int phil_id[N] = {0,1,2,3,4}; // Array of philosopher IDs (passed to threads)

sem_t mutex;           // Binary semaphore: protects the state[] array
sem_t S[N];            // One semaphore per philosopher: blocks them when they can't eat

// — test(): Try to move philosopher i to EATING state —————
void test(int i) {
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING) {
        // Philosopher i can eat if:
        // 1. They are HUNGRY (wanting to eat)
        // 2. Left neighbor is NOT eating (left fork is available)
        // 3. Right neighbor is NOT eating (right fork is available)

        state[i] = EATING;
        printf("Philosopher %d takes forks %d and %d → EATING\n",
            i, LEFT(i), i);
        sem_post(&S[i]);
        // sem_post on philosopher i's personal semaphore
        // This UNBLOCKS philosopher i if they were waiting in take_forks()
    }
}

// — take_forks(): Philosopher i tries to pick up both forks —————
void take_forks(int i) {
    sem_wait(&mutex);
    // Acquire global mutex to safely modify state[]

    state[i] = HUNGRY;
    printf("Philosopher %d is HUNGRY\n", i);

```

```

    test(i);
    // Check if philosopher i can start eating right now

    sem_post(&mutex);
    // Release global mutex regardless of outcome

    sem_wait(&S[i]);
    // If test() succeeded → S[i] was already sem_post'd → this returns immediately
    // If test() failed → S[i] is 0 → philosopher BLOCKS here until
    //                      a neighbor calls put_forks() and unblocks them
}

// — put_forks(): Philosopher i puts down both forks —————
void put_forks(int i) {
    sem_wait(&mutex);

    state[i] = THINKING;
    printf("Philosopher %d puts down forks %d and %d → THINKING\n",
        i, LEFT(i), i);

    test(LEFT(i));
    // Check if left neighbor can now eat (they might have been waiting)

    test(RIGHT(i));
    // Check if right neighbor can now eat

    sem_post(&mutex);
    // Release forks effectively by updating state and testing neighbors
}

// — philosopher(): Thread function for each philosopher —————
void *philosopher(void *arg) {
    int i = *(int *)arg;
    // Cast void* argument back to int*, then dereference

    while (1) {
        printf("Philosopher %d is THINKING\n", i);
        sleep(2);           // Think for 2 seconds
        take_forks(i);      // Try to pick up forks (may block)
        sleep(1);           // Eat for 1 second
        put_forks(i);       // Put forks down, allow neighbors to eat
    }
    return NULL;
}

// — main() —————

```

```

int main() {
    pthread_t threads[N];

    sem_init(&mutex, 0, 1);
    // mutex starts at 1 (unlocked binary semaphore)

    for (int i = 0; i < N; i++) {
        sem_init(&S[i], 0, 0);
        // Each philosopher's semaphore starts at 0 (BLOCKED)
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&threads[i], NULL, philosopher, &phil_id[i]);
        // Pass address of phil_id[i] so the thread knows its own ID
    }

    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
        // Wait for all philosopher threads (runs forever in practice)
    }

    // Cleanup
    sem_destroy(&mutex);
    for (int i = 0; i < N; i++) sem_destroy(&S[i]);

    return 0;
}

```

Compile & Run:

```

bash

gcc q7_dining.c -o q7 -lpthread
./q7

```

Why this solution avoids deadlock:

- Naive solution: everyone picks up left fork simultaneously → circular wait → deadlock
- This solution: before picking up forks, philosopher atomically checks BOTH neighbors via `test()`. They only enter EATING if both forks are actually free. If not, they block on their personal semaphore until a neighbor wakes them.

Q8: Banker's Algorithm — Safe/Unsafe State

```

#include <stdio.h>
#include <stdbool.h>    // bool, true, false

#define MAX_P 10    // Maximum processes
#define MAX_R 10    // Maximum resource types

int main() {
    int n, m;
    // n = number of processes, m = number of resource types

    int alloc[MAX_P][MAX_R];    // Allocation matrix: alloc[i][j] = resources of type j allocated to process i
    int maxm[MAX_P][MAX_R];    // Max matrix: maxm[i][j] = max resources of type j for process i
    int avail[MAX_R];    // Available vector: avail[j] = currently available units of resource type j
    int need[MAX_P][MAX_R];    // Need matrix: need[i][j] = maxm[i][j] - alloc[i][j]
    bool finish[MAX_P];    // finish[i] = true when process i has completed
    int safeSeq[MAX_P];    // Stores the safe sequence
    int work[MAX_R];    // Working copy of available (simulates giving resources to processes)

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);

    printf("\nEnter Allocation Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }

    printf("\nEnter Max Demand Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &maxm[i][j]);
    }

    printf("\nEnter Available Resources (1 x %d): ", m);
    for (int j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    // — STEP 1: Calculate Need matrix —————
    printf("\n— Need Matrix (Max - Allocation) —\n");
    for (int i = 0; i < n; i++) {
        finish[i] = false;
        // Initialize all processes as unfinished
    }
}

```

```

printf("P%d: ", i);
for (int j = 0; j < m; j++) {
    need[i][j] = maxm[i][j] - alloc[i][j];
    // need[i][j] = how many MORE resources of type j process i could request
    printf("%d ", need[i][j]);
}
printf("\n");
}

// — STEP 2: Initialize work = available —————
for (int j = 0; j < m; j++)
    work[j] = avail[j];
// work[] = simulated available resources as we "run" processes

// — STEP 3: Safety Algorithm (find safe sequence) —————
int count = 0;    // How many processes have been added to safe sequence

while (count < n) {
    bool found = false;

    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            // Process i hasn't completed yet

            bool canAllocate = true;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    // Process i needs MORE of resource j than currently available
                    // → Cannot safely allocate to this process right now
                    canAllocate = false;
                    break;
                }
            }

            if (canAllocate) {
                // Pretend to run process i to completion:
                // It gets what it needs, runs, then releases ALL its resources
                for (int j = 0; j < m; j++)
                    work[j] += alloc[i][j];
                // Returned resources go back into work pool

                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
                printf("Process P%d added to safe sequence. Work: ", i);
                for (int j = 0; j < m; j++) printf("%d ", work[j]);
                printf("\n");
            }
        }
    }
}

```

```

        }
    }
}

if (!found) {
    // Full pass through all processes with nothing found → UNSAFE
    break;
}

// — STEP 4: Result —————
printf("\n=====\\n");
if (count == n) {
    printf("System is in SAFE STATE.\\nSafe Sequence: < ");
    for (int i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);
    printf(">\\n");
} else {
    printf("System is in UNSAFE STATE.\\n");
    printf("Deadlock may occur!\\n");
}

return 0;
}

```

Compile & Run:

```
bash
```

```
gcc q8_bankers.c -o q8
./q8
```

Sample Input:

```
Processes: 5, Resources: 3
```

```
Allocation:    Max:
```

```
P0: 0 1 0      P0: 7 5 3
```

```
P1: 2 0 0      P1: 3 2 2
```

```
P2: 3 0 2      P2: 9 0 2
```

```
P3: 2 1 1      P3: 2 2 2
```

```
P4: 0 0 2      P4: 4 3 3
```

```
Available: 3 3 2
```

```
Expected Safe Sequence: <P1 P3 P4 P0 P2>
```

Q9: Page Replacement — FIFO

c

```

#include <stdio.h>
#include <stdbool.h>

int main() {
    int frames, n;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter length of reference string: ");
    scanf("%d", &n);

    int ref[n];
    printf("Enter reference string: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    int page[frames];          // Current pages held in frames
    for (int i = 0; i < frames; i++)
        page[i] = -1;          // -1 = empty frame

    int pointer = 0;           // Points to the OLDEST page (next to be replaced)
    int pageFaults = 0;

    printf("\nRef | Frames                                | Fault?\n");
    printf("-----+-----+-----\n");

    for (int i = 0; i < n; i++) {
        bool hit = false;

        // — Check if page is already in a frame (page hit) —
        for (int j = 0; j < frames; j++) {
            if (page[j] == ref[i]) {
                hit = true;
                break;
            }
        }

        printf(" %2d | ", ref[i]);

        if (!hit) {
            // PAGE FAULT: page not in memory → must load it
            page[pointer] = ref[i];
            // Replace the frame pointed to by 'pointer' (FIFO: oldest first)

            pointer = (pointer + 1) % frames;

```



```

// Advance pointer circularly
// FIFO: replace in round-robin order → oldest page is always at 'pointer'

pageFaults++;

for (int j = 0; j < frames; j++) {
    if (page[j] == -1) printf(" - ");
    else                printf(" %2d ", page[j]);
}
printf("| FAULT\n");
} else {
    // PAGE HIT: no replacement needed
    for (int j = 0; j < frames; j++) {
        if (page[j] == -1) printf(" - ");
        else                printf(" %2d ", page[j]);
    }
    printf("| hit\n");
}
}

printf("\nTotal Page Faults (FIFO): %d\n", pageFaults);
return 0;
}

```

FIFO Key Idea: Maintain a circular queue of frames. When a fault occurs, evict the page that has been in memory the LONGEST (i.e., arrived first). The pointer variable tracks the next-to-evict position.

Note: Bélády's Anomaly — FIFO can have MORE faults with MORE frames (counterintuitive). LRU and OPT don't suffer from this.

Q10: Page Replacement — LRU

c

```

#include <stdio.h>
#include <stdbool.h>
#include <limits.h>      // INT_MIN

int main() {
    int frames, n;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter length of reference string: ");
    scanf("%d", &n);

    int ref[n];
    printf("Enter reference string: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    int page[frames];      // Pages currently in frames
    int lastUsed[frames];  // Timestamp of last use for each frame slot

    for (int i = 0; i < frames; i++) {
        page[i]      = -1;    // Empty frame
        lastUsed[i] = -1;    // Never used
    }

    int pageFaults = 0;

    printf("\nRef | Frames                                | Fault?\n");
    printf("-----|-----|-----\n");

    for (int i = 0; i < n; i++) {
        bool hit      = false;
        int  freeSlot = -1;
        int  hitIdx   = -1;

        // — Check for hit and find free slot —————
        for (int j = 0; j < frames; j++) {
            if (page[j] == ref[i]) {
                hit      = true;
                hitIdx = j;
                break;
            }
            if (page[j] == -1 && freeSlot == -1)
                freeSlot = j;
        }
        // Note the first free (empty) slot found
    }

```

```

}

printf(" %2d | ", ref[i]);

if (hit) {
    // PAGE HIT: update the last-used timestamp for this page
    lastUsed[hitIdx] = i;
    // 'i' is the current time step (reference string position)

    for (int j = 0; j < frames; j++) {
        if (page[j] == -1) printf(" - ");
        else                printf(" %2d ", page[j]);
    }
    printf("| hit\n");
} else {
    // PAGE FAULT: must bring page in
    pageFaults++;
    int replaceIdx;

    if (freeSlot != -1) {
        // There's still an empty frame – use it directly (no eviction)
        replaceIdx = freeSlot;
    } else {
        // All frames full → find LRU: page with the SMALLEST lastUsed value
        int minTime = INT_MAX, minIdx = 0;
        for (int j = 0; j < frames; j++) {
            if (lastUsed[j] < minTime) {
                minTime = lastUsed[j];
                minIdx = j;
            }
        }
        // minIdx = frame holding the Least Recently Used page
        replaceIdx = minIdx;
    }

    page[replaceIdx] = ref[i];
    lastUsed[replaceIdx] = i;
    // Load new page and record its usage time

    for (int j = 0; j < frames; j++) {
        if (page[j] == -1) printf(" - ");
        else                printf(" %2d ", page[j]);
    }
    printf("| FAULT\n");
}
}

```

```
printf("\nTotal Page Faults (LRU): %d\n", pageFaults);  
return 0;  
}
```

LRU Key Idea: When a fault occurs, evict the page whose last access was furthest in the past. The `lastUsed[]` array tracks the timestamp (index in reference string) of the most recent access per frame.

LRU vs FIFO:

- LRU is generally better (exploits temporal locality)
 - LRU never suffers Bélády's Anomaly
 - LRU is more expensive to implement (hardware support needed in real OS)
-

Q11: CPU Scheduling — FCFS (First Come First Served)

c

```

#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n];
    // at = Arrival Time, bt = Burst Time (CPU time needed)

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("P%d → Arrival Time, Burst Time: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    // — Sort by Arrival Time (Bubble Sort) —————
    // FCFS serves processes in the order they arrive
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                int t;
                // Swap at
                t = at[j]; at[j] = at[j+1]; at[j+1] = t;
                // Swap bt (must move together with at)
                t = bt[j]; bt[j] = bt[j+1]; bt[j+1] = t;
                // Swap pid
                t = pid[j]; pid[j] = pid[j+1]; pid[j+1] = t;
            }
        }
    }

    int ct[n], tat[n], wt[n];
    // ct = Completion Time (when process finishes)
    // tat= Turnaround Time = CT - AT (total time from arrival to finish)
    // wt = Waiting Time = TAT - BT (time spent waiting, not executing)

    int time = 0;
    // 'time' is our CPU clock

    for (int i = 0; i < n; i++) {
        if (time < at[i])
            time = at[i];
        // If CPU is idle (no process yet), jump clock to arrival of next process

        ct[i] = time + bt[i];
    }
}

```

```

        // Completion = current time + burst time (runs to completion in FCFS)

        tat[i] = ct[i] - at[i];
        // Turnaround = Completion - Arrival

        wt[i] = tat[i] - bt[i];
        // Waiting = Turnaround - Burst
        // (time spent NOT running = turnaround time minus actual execution time)

        time = ct[i];
        // Advance clock to when this process finishes
    }

    // ——— Print Table ———
    printf("\nProcess | AT | BT | CT | TAT | WT\n");
    printf("—————+—————+—————+—————+—————+—————\n");

    float totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) {
        printf("  P%-4d | %2d | %2d | %2d |  %2d | %2d\n",
            pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
        totalWT += wt[i];
        totalTAT += tat[i];
    }

    printf("\nAverage Waiting Time      : %.2f\n", totalWT / n);
    printf("Average Turnaround Time : %.2f\n", totalTAT / n);

    return 0;
}

```

Sample Input:

```

P1: AT=0, BT=5
P2: AT=1, BT=3
P3: AT=2, BT=8

```

Sample Output:

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

Average Waiting Time : 3.33

Average Turnaround Time : 8.67

FCFS Analysis:

- Simple, fair, no starvation
- Suffers from **Convoy Effect**: short processes stuck behind long ones → high average WT
- Non-preemptive: once a process starts, it runs until done

Q12: CPU Scheduling — SJF (Shortest Job First, Non-Preemptive)

c

```

#include <stdio.h>
#include <stdbool.h>
#include <limits.h>    // INT_MAX

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];
    bool done[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        done[i] = false;
        printf("P%d → Arrival Time, Burst Time: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    int time = 0;
    int completed = 0;
    float totalWT = 0, totalTAT = 0;

    printf("\nProcess | AT | BT | CT | TAT | WT\n");
    printf("-----|-----|-----|-----|-----\n");

    while (completed < n) {
        // — Find eligible process with shortest burst time —
        int minBT = INT_MAX;
        int idx = -1;

        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                // Process i is eligible if:
                // 1. Not yet completed
                // 2. Has already arrived (at[i] <= current time)

                if (bt[i] < minBT) {
                    minBT = bt[i];
                    idx = i;
                    // Track process with smallest burst time
                }
            }
        }

        if (idx == -1) {

```



```

        // No eligible process found at current time → CPU is idle
        time++;
        // Advance time by 1 and try again
        continue;
    }

    // — Schedule the selected (shortest) process —————
    ct[idx] = time + bt[idx];
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    time = ct[idx];
    done[idx] = true;
    completed++;

    totalWT += wt[idx];
    totalTAT += tat[idx];

    printf(" P%-4d | %2d | %2d | %2d | %2d | %2d\n",
           pid[idx], at[idx], bt[idx], ct[idx], tat[idx], wt[idx]);
}

printf("\nAverage Waiting Time      : %.2f\n", totalWT / n);
printf("Average Turnaround Time : %.2f\n", totalTAT / n);

return 0;
}

```

Compile & Run:

```

bash

gcc q12_sjf.c -o q12
./q12

```

Sample Input: Same as FCFS example above (P1: 0,5 | P2: 1,3 | P3: 2,8)

SJF Output:

```

P2  | AT=1 | BT=3 | CT=8  | TAT=7  | WT=4
P1  | AT=0 | BT=5 | CT=13 | TAT=13 | WT=8
P3  | AT=2 | BT=8 | CT=21 | TAT=19 | WT=11

Average Waiting Time      : 7.67    ← HIGHER than FCFS here due to arrival times
Average Turnaround Time  : 13.00

```

Note: SJF is optimal when all processes arrive at time 0. With different arrival times, the advantage depends on the dataset.

SJF Analysis:

- Minimum average waiting time when all processes arrive simultaneously
- Can cause **starvation**: a very long process may wait indefinitely if short ones keep arriving
- Preemptive variant = SRTF (Shortest Remaining Time First)

Compilation Quick Reference

```
bash

# Shell scripts
chmod +x script.sh && ./script.sh

# C programs (no threading)
gcc filename.c -o output_name
./output_name

# C programs with pthreads (Q5, Q6, Q7)
gcc filename.c -o output_name -lpthread
./output_name
```

Concept Summary Table

Topic	Q#	Key System Call / Primitive
Shell palindrome + factorial	Q1	rev, \$(())
File stats + palindrome	Q2	wc, pipes
File I/O system calls	Q3	open, read, write, lseek, unlink
Process creation	Q4	fork, wait, getpid
Producer-Consumer (count)	Q5	sem_wait/post, pthread_mutex
Producer-Consumer (clean)	Q6	sem_wait/post, pthread_mutex
Dining Philosophers	Q7	sem_wait/post, pthread_create
Banker's Algorithm	Q8	Need = Max – Alloc; Safety loop
FIFO Page Replacement	Q9	Circular pointer replacement

Topic	Q#	Key System Call / Primitive
LRU Page Replacement	Q10	<code>lastUsed[]</code> timestamp tracking
FCFS Scheduling	Q11	CT, TAT = CT-AT, WT = TAT-BT
SJF Scheduling	Q12	Min burst time among arrived processes